KNN Homework: From Measurements to Neighbor Votes

Format: Individual take-home investigation in your own Git repository
Estimated effort: 4-6 hours, plus documentation and independent peer validation
Prerequisite: The KNN beginner guide and guided comparison notebook

Purpose

Show that you can explain what KNN stores, prepare measurements without leaking answers, calculate a prediction, select k using validation data, and interpret errors on previously unused test data. Correct code is necessary, but an output table alone does not demonstrate understanding.

This homework continues the course's professional portfolio and independent replication expectations. It is separate from the logistic regression project; you do not need to implement gradient descent, logistic regression, a decision tree, an API, or Docker.

Materials and setup

Review the student guide and guided notebook. Use these as references, not as a completed homework submission.

Use Python 3.10 or newer with NumPy, Matplotlib, and scikit-learn. The existing course dependency file supplies these libraries. From the course repository root in Windows PowerShell:

```
python -m pip install -r .\Project_02_Classification\assignment_requirements.txt
```

In macOS Terminal, use `python3` and `Project_02_Classification/assignment_requirements.txt`. Your own repository must include its own dependency file and complete setup instructions. No separate dataset download is needed after installing scikit-learn.

Part 1: Explain the algorithm before calculating

Answer in your own words:

1. What are an input feature, a known target label, a query, and k? Does k specify the number of species?
2. Distinguish what `StandardScaler.fit` learns from what `KNeighborsClassifier.fit` stores. Which step uses species labels?
3. What determines which flowers vote? Why must we sort by distance before taking k neighbors? Why do the other reference flowers not vote?
4. Does ordinary KNN repeatedly update coefficients to minimize a loss? Explain when fitting ends, when choosing k ends, and when one prediction ends.
5. Can a classifier trained on three named species discover an unknown fourth species automatically? Suggest a different exploratory approach and explain why its groups would still require expert review.

Part 2: Complete the entire pipeline by hand

These are **six invented teaching records**, not observations from the real Iris dataset. Treat all six as the training reference set for this part only. L means petal length and W means petal width, both in centimeters. The ID is for tracking records, not an input feature.

ID	L (cm)	W (cm)	Known species
A	2.0	1.0	setosa
B	2.0	2.0	setosa
C	4.0	1.0	versicolor
D	4.0	2.0	versicolor
E	6.0	1.0	virginica
F	6.0	2.0	virginica

The new query Q has **L = 4.7 cm and W = 1.8 cm**. Its true species is unknown. Do not include Q when computing training statistics.

A. Calculate the training statistics

For each feature separately, show:

1. The sum and mean of its six training values.
2. Each deviation from the mean and each squared deviation.
3. The sum of squared deviations, the population variance, and its square root.

Use the **population** standard deviation, matching `StandardScaler`:

```
mean = sum of the six training values / 6
variance = sum of (training value - mean)^2 / 6
standard deviation = square root of variance
```

Do not divide by 5. Retain unrounded values in subsequent calculations and display final numbers to four decimal places.

B. Scale, calculate distances, sort, then vote

Use the same saved training statistics for A-F and Q:

```
z_L = (L - training mean of L) / training standard deviation of L
z_W = (W - training mean of W) / training standard deviation of W
```

Submit a table with all seven records' scaled length and width. Show the substitution arithmetic for Q and at least one training record.

For each training reference, calculate its Euclidean distance to Q:

```
distance = square root of [(z_L_reference - z_L_Q)^2
                         + (z_W_reference - z_W_Q)^2]
```

1. Show every subtraction, square, sum, and square root for reference D.
2. Submit all six distances, sorted from closest to farthest, with IDs and labels.
3. Predict Q's species for k = 1 and k = 3. Name the voters and count each species' votes for each prediction.
4. For k = 3, calculate the winning vote fraction and percentage. Explain what the percentage describes, why it is not guaranteed probability of being correct, and whether it is fed back into a coefficient update.
5. Calculate all six distances again using **raw centimeters**, without scaling. Compare the neighbor order and the k = 3 prediction with the scaled calculation. Explain why the order can change even when both features use cm.

6. Explain why training means and standard deviations prepare the measurements but cannot replace distance calculations or species voting. Could you report accuracy for Q without its true label?
7. Can odd k still produce a tied vote with three classes? Give a possible vote-count example and explain why an implementation needs a tie rule.

Complete and show your own arithmetic first. You may then check it with Python. For this check, write the mean, population variance, scaling, and distance calculations yourself using Python or NumPy arithmetic; do not use `StandardScaler`, `KNeighborsClassifier`, or a library nearest-neighbor search to replace your Part 2 work. No from-scratch full-dataset classifier is required.

Part 3: Design a leakage-free Iris experiment

This part uses **all 150 real observations** from `sklearn.datasets.load_iris()`, not the invented Part 2 records. Read `DESCR` and acknowledge the dataset source in your report.

Use exactly these input features, in this order:

1. `petal length (cm)` (column 2).
2. `petal width (cm)` (column 3).

Keep the original targets: **0 = setosa, 1 = versicolor, 2 = virginica**. Never use species codes or row IDs as distance features; do not average species codes to make predictions.

1. Report dataset size, feature names, missing-value counts, and class counts.
2. Split row indices with `train_test_split` in this exact sequence:
   - Development/test: `test_size=0.20`, `stratify=y`, `random_state=42`.
   - Training/validation within development: `test_size=0.25`, `stratify=y[development_indices]`, `random_state=42`.
   - Keep returned index order; do not sort or reshuffle it.
3. Print row counts and class counts in all three partitions. Verify that their indices are pairwise disjoint and together cover all 150 rows.
4. State each partition's purpose. Do not inspect test performance or use test measurements to fit preprocessing during development.

Do not combine this experiment with the notebook's breast-cancer results: the dataset, species labels, and metric definitions here are different.

Part 4: Select k and explain overfitting

You may use scikit-learn for this part. For each k in **[1, 3, 5, 7, 9, 15, 25]**, create a pipeline containing:

- `StandardScaler()`.
- `KNeighborsClassifier(n_neighbors=k, weights="uniform", metric="minkowski", p=2, algorithm="brute")`.

Fit each pipeline on the **training partition only**, then record training and validation accuracy. The scaler must not be fitted separately on validation. Keep all other settings and all partitions identical across candidates.

1. Export one row per k and plot both accuracies against k with labeled axes and a legend.
2. Select the k with the highest **validation accuracy**. Break exact score ties by choosing the **smallest k**. Compare unrounded scores or integer correct counts, not rounded display values. Record the selected k before opening the test results.

3. Explain why training accuracy does not select k. Why can k = 1 look perfect when predicting its own training rows? Interpret the observed gaps and whether they suggest overfitting or underfitting; do not assume a pattern that your results do not show. A small gap alone is not proof of a good model.
4. Does searching these seven candidates mean KNN made seven gradient updates? Explain what actually changed and why the search ends.
5. Explain how your library handles equal class votes. A distance tie at the kth-neighbor boundary is a different issue; acknowledge that equal distances, row ordering, and library versions can affect which tied reference is selected.

Part 5: Freeze, refit, and evaluate once

After freezing k, create a **new pipeline with that k** and fit both its scaler and KNN on the combined training and validation rows. Use the original development-index order from Part 3. Do not reuse the training-only scaler as though it were fitted on the combined data.

Fit `DummyClassifier(strategy="most_frequent")` on those same development rows as a baseline. No scaling is needed for this baseline.

Only now evaluate both models on the held-out test partition in one final stage. Do not change k, features, scaling, or other settings in response to test results. The Part 4 scores describe the training-only fits; do not present them as scores of the refitted model.

For both models:

- Produce a confusion matrix with `labels=[0, 1, 2]`, rows = actual and columns = predicted, labeled with species names.
- Report accuracy and **macro-F1**.
- Report each species' precision, recall, F1, and support (actual test count). Use `zero_division=0` when a metric denominator is zero and explicitly explain that a displayed zero can represent an undefined metric for the baseline.

Explain accuracy and macro-F1 in plain language. Interpret at least one off-diagonal confusion count as a real classification mistake. Compare KNN with the baseline using your actual results. Do not claim that a high score establishes field performance, causation, or automatic recognition of new species. Explain why the final refit may change predictions even when k stays fixed.

Required generated outputs

Write these files inside your own repository. Create `reports/` if needed.

File	Required content
`reports/knn_validation.csv`	Seven rows; columns `k`, `train_accuracy`, `validation_accuracy`
`reports/test_summary.csv`	Two rows; columns `model`, `accuracy`, `macro_f1`; model values `knn` and `baseline`
`reports/test_per_species.csv`	Six rows; columns `model`, `species`, `precision`, `recall`, `f1`, `support`; each species for each model
`reports/k_selection.png`	Training and validation accuracy versus k
`reports/test_confusion_matrices.png`	Both test confusion matrices, clearly titled and labeled

Use species names `setosa`, `versicolor`, and `virginica` in the per-species CSV. Save metrics as decimals in [0, 1] with at least six decimal places, not percentages; k and support are integers. Print the selected k and readable result tables as

well. Make output paths relative to your script's location so execution does not depend on the terminal's current directory.

Part 6: Interpret supplied errors and large-data costs

A. Metrics by hand

The following matrix is **hypothetical** and separate from your code results. Rows are actual species; columns are predicted species.

Actual / Predicted	setosa	versicolor	virginica
setosa	18	2	0
versicolor	1	15	4
virginica	0	3	17

1. Calculate total predictions, correct predictions, errors, and accuracy.
2. Treat virginica as positive and all other species as negative. Identify TP, FP, FN, and TN. Calculate virginica precision, recall, and F1, showing formulas and substitutions.
3. Interpret precision and recall using their actual denominators: "Of flowers predicted to be virginica..." versus "Of actual virginica flowers...". Describe one consequence of each kind of error.
4. Explain why virginica F1 is neither overall accuracy nor macro-F1. Do not merge this matrix into your experimentally measured results.

B. An 85-million-row reasoning exercise

Assume 85 million labeled, distinct plants: 51 million training, 17 million validation, and 17 million test. You display only five rows for illustration. All rows remain in their assigned roles; no huge dataset or run is required.

1. How many references are eligible for each validation query? After a final training-plus-validation refit, how many are eligible for each test query? Why must all 85 million not vote during validation?
2. For one exhaustive validation pass, calculate the number of query-reference distance comparisons. Explain how retaining the nearest 25 references per query could let you score all seven Part 4 candidates without seven full scans.
3. With 10,000 validation queries per batch, how many batches are needed? Does each query search only its batch, or the full eligible reference set? Does batching reduce the total exhaustive comparison count?
4. Distinguish query batching, chunked scaler-statistic computation, and gradient-descent minibatches. Explain why fitting a separate scaler per chunk or repeatedly calling KNN `fit` on successive chunks is not equivalent to one model retaining all references. KNN has no `partial_fit`.
5. If references are searched in blocks, what must be combined across blocks before taking a vote? Why is voting only within the last block wrong?
6. Is there one universal KNN row limit? Distinguish eligible references from rows actually examined by an exact search index. Explain one trade-off of approximate-neighbor search and what you would validate before using it.

Written report and code deliverables

Submit `memo.md` or `memo.pdf` with Parts 1, 4-6 interpretations, about **700-1,000 words excluding calculations and tables**. Include Part 2's complete hand calculations and tables, Part 6A's calculations, your chosen k and why it won, and tables or links to the required measured results. Keep invented and measured results clearly separated. Legible scans of your handwritten work may be included or linked; ensure the instructor can read them.

Add a short research-process section outside the suggested word count: describe a real difficulty or design decision, how you investigated it, and the evidence you used to check the outcome. Do not invent problems. Acknowledge sources and reused code, including course materials and any tools used to assist your work. You must be able to explain your submitted work.

Include one end-to-end Python script, such as `knn_analysis.py`, that performs Parts 3-5 and regenerates all five required outputs. If you use Python to check Part 2, include that check in the script or a clearly documented separate file. A notebook is optional and cannot replace the runnable script.

Professional portfolio repository

Use your own Git repository with a descriptive title such as **Iris Species Classification: Scaling, Neighbors, and Validation**, not a generic homework number. Include:

- Your script, dependency file, report/calculations, and all required outputs.
- A self-contained root `README.md` explaining the question, data source, feature units, species codes, workflow, selected k, findings, and limitations.
- Logical organization, meaningful commits, and a `.gitignore` excluding virtual environments, caches, secrets, and the completed peer-validation form.

Do not commit a virtual environment, a copied Iris dataset, unrelated course materials, signatures, or private reviewer information.

README: a beginner must be able to reproduce your work

Assume the reader has never used Python, Git, or a terminal. Do not assume VS Code or other software is already installed. Provide:

1. Python and Git installation links, how to verify installation, and how to open Windows PowerShell and macOS Terminal.
2. Your actual repository URL, exact clone and navigation commands, and an explanation of any placeholders.
3. Separate Windows PowerShell and macOS instructions for creating and activating an environment, installing dependencies, and running your script. Include a direct environment-interpreter alternative if PowerShell activation is blocked; do not require a permanent security-policy change.
4. Commands with their working directories, tested Python/package versions, and the fact that Iris is bundled in scikit-learn.
5. Expected output names/locations, how to open them, approximate runtime on your machine, and reasonable floating-point or equal-distance tie variation.
6. Troubleshooting for an unrecognized Python command, missing dependencies, and execution from the wrong directory.

Your README must not require private messages, your computer's absolute paths, or access to local instructor files. A separate setup note cannot replace it.

Independent peer validation and Canvas submission

Use the course Project Replication & Peer Verification Form. The form's replication policy, including its stated deductions, applies.

1. Send a classmate the repository URL and branch/version. They must obtain a fresh copy on their own computer and follow **only your README**, without questions, live guidance, remote control, or other help from you.
2. They check setup, successful execution, and regeneration of every output. They record their OS, Python version, branch, final commit identifier, and any blockers. Attach notes to the form if necessary.

3. If anything fails or is unclear, record the issue, fix code/documentation, and repeat independent validation on the updated version before signing. Record the final successful commit. Do not claim both platforms were tested unless they actually were; instructions for both are still required.
4. Both parties complete the form honestly. Keep signatures and private reviewer details **out of Git**.

Submit on Canvas:

- The accessible Git repository link, branch, and final independently validated commit identifier.
- The signed validation form as a PDF, with any verification notes, uploaded privately to Canvas.
- A short note identifying any optional extensions and where to find them, or stating that you completed the required scope only.

A ZIP, screenshot, or script upload is not a substitute for the Git link and validation PDF. Keep extensions separate from the prescribed experiment. Additional tuning must not use the held-out test results.

Use the student rubric before submitting. You are assessed on correct reasoning, reproducibility, and honest interpretation, not a required accuracy, a particular winning k, or beating the baseline on every metric.